

Setting Up Your Developer Environment

The unglamorous setup that makes everything else possible. Virtual environments, VS Code, Jupyter inside VS Code, Git, GitHub and your first green push. Get this right once and you stop fighting your machine — forever.

WHAT'S INSIDE

- 01** Virtual environments (venv)
- 02** requirements.txt the right way
- 03** VS Code for AI work
- 04** Jupyter inside VS Code
- 05** Git, GitHub & your first push

WHY ENVIRONMENT FIRST

90% of student pain is environment, not maths

ROOT CAUSE

90%

Of debugging time spent on environment issues — not the actual code.

THIS LESSON

40 min

To eliminate that entire class of problem, forever.

NON-NEGOTIABLE

1 push

Every learner ends with a green 'push successful' on a public repo.

WHY IT MATTERS

forever

You will set up new projects 50+ times in your career. Get the rhythm right today.

LEARNING OUTCOMES

By the end of this lesson, you will be able to —

1

Create and manage a Python virtual environment per project.

2

Pin and freeze dependencies with requirements.txt.

3

Configure VS Code with the essential extensions for AI work.

4

Run Jupyter notebooks inside VS Code, not in a separate browser.

5

Use Git for version control and GitHub for collaboration.

6

Resolve a merge conflict calmly — without panic.

AGENDA

What we'll cover today

Short, sequenced parts. Each one builds on the last.

01

Three pillars

Jupyter for exploration, VS Code for engineering, Git for memory. Each one earns its place.

02

venv & requirements

One environment per project. Pin everything. Reproducible setups every time.

03

VS Code, configured

Extensions that matter, settings that save hours, shortcuts that compound.

04

Git → GitHub → green

From git init on your laptop to a green checkmark on a public repo — the actual mechanics.

01

PART 1 OF 4

The three pillars — and what each is best at

Each tool has a job. Pick the wrong one and the work fights you all day.

ROLE EACH TOOL PLAYS

What each pillar is actually for

Don't write production code in a notebook. Don't explore data in a CLI. Use the right tool for the moment.



JUPYTER

For exploring data

Cells, plots, narratives. Best when you don't yet know what the answer looks like. Worst for production code.



VS CODE

For building software

Editor + debugger + terminal + Git, all in one window. The home of every refactor, every test, every package.



GIT + GITHUB

For memory & teamwork

Git is your time machine — every change reversible. GitHub turns one engineer's work into a team's work.

02

PART 2 OF 4

Virtual environments — one per project, no exceptions

The five-minute habit that prevents 90% of dependency disasters.

THE COMMANDS YOU'LL RUN 100 TIMES

The four commands that set up any Python project

Memorise these. You will type them on every new project, every machine, for the rest of your career.

```
>>> PYTHON

# 1. Create a fresh venv inside your project folder
python -m venv .venv

# 2. Activate it (macOS/Linux)
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# 3. Install your packages
pip install pandas scikit-learn jupyter

# 4. Freeze them so anyone can reproduce your setup
pip freeze > requirements.txt
```

→ One venv per project. Never install with `sudo pip`. Never use the system Python directly.

03

PART 3 OF 4

VS Code, configured for real work

Extensions that pay rent, settings that compound, shortcuts that save hours.

THE ESSENTIAL EXTENSIONS

Six extensions to install before you write a line

Don't install fifty extensions. Install these six, configure them once, then forget the marketplace exists.



PYTHON

Microsoft Python

Linting, IntelliSense, debugging, env detection. Non-optional.



JUPYTER

Jupyter

Run .ipynb files inside VS Code. Same kernel, same plots, no browser.



GITLENS

GitLens

Annotates each line with who changed it, when, and why. Reviews become 5x faster.



RUFF

Ruff

Linters + formatters, written in Rust. Catches issues before tests do — instantly.



ERROR LENS

Error Lens

Surfaces errors and warnings inline, on the same line. No more hunting.



COPILOT (opt)

Copilot or Claude

AI pair-programmer for boilerplate, tests, docstrings. Useful — but always review the diff.

WHEN TO LEAVE THE NOTEBOOK

Notebooks are for exploring. Modules are for keeping.

Notebooks shine when you don't know the answer yet — plotting, fiddling, narrating. But notebooks fall apart when:

- the same code lives in three cells
- cell-execution order matters and you've lost track
- someone else has to read it next Tuesday

When any of these starts hurting, extract the stable bits into a .py module and import it back into the notebook.

FOUR RULES OF THUMB

- Use a notebook for the first 200 lines of a problem.
- Extract pure functions into a .py file as soon as they stabilise.
- Import that .py back into the notebook — keep the narrative.
- Commit notebooks with cleared outputs. Diffs become readable.

04

PART 4 OF 4

Git, GitHub, and your first green push

By the end of this part, you will have a public repo and a working pull request.

Six commands. Five minutes. Your first public push.

Run these in order. The non-negotiable end state is a green 'push successful' on a public GitHub repo.



✦ THINK • PAIR •
SHARE

ACTIVITY

Match each activity to the right tool

PROMPT

Drop each task below into the right column — Jupyter, VS Code or Git.

- Writing and testing a Python package
- Plotting six versions of a histogram while exploring data
- Reverting a commit that broke production
- Debugging a script with breakpoints and watch variables
- Reviewing a teammate's changes line-by-line
- Telling a story about a dataset, with code and prose interleaved

HOW TO RUN IT

- 1 60s solo — sort each activity into a tool.
- 2 60s with a partner — defend any borderline picks.
- 3 60s share-out — instructor calls on one team.
- 4 30s reveal — instructor reveals the canonical sort.

WORKED EXAMPLE

Setting up a 3-person Indian student project — the right way

THE SCENARIO

Three students in Pune are starting an end-of-semester ML project. Two are on Mac, one on Windows. They will share a private GitHub repo. They have one week. The previous batch's project failed during demo because 'it works on my laptop' — the code wouldn't run on the evaluator's machine.

THE TAKEAWAY

One venv each (.venv per machine), one requirements.txt in the repo, notebooks in /notebooks/ with cleared outputs, package code in /src/, tests in /tests/. The evaluator runs three commands and everything works.

WHAT TO NOTICE

- 1 Each developer creates their own venv — they are local, never committed.
- 2 requirements.txt is committed; .venv is in .gitignore.
- 3 Notebooks live in /notebooks/, with outputs cleared before commit.
- 4 Reproducibility is one README away: clone → venv → install → run.

Your teammate sends you a project. They forgot the requirements.txt. What's the senior-engineer move?

A

Install whatever pip suggests on each ImportError

B

Delete your venv and use the system Python — easier

C

Ask them to run `pip freeze > requirements.txt` and re-share

D

Skip the venv altogether for this project

Your teammate sends you a project. They forgot the requirements.txt. What's the senior-engineer move?



CORRECT ANSWER

Ask them to run `pip freeze > requirements.txt` and re-share

WHY

Reproducibility is non-negotiable. Without a pinned requirements file, you'll spend hours guessing versions. The right answer is the small, polite request that saves the next week.

When should you stop developing in a Jupyter notebook?

A

Never — notebooks are always preferred

B

When the notebook crosses ~200 lines or the cell order starts mattering

C

Only when the project is already in production

D

When your kernel crashes more than once

When should you stop developing in a Jupyter notebook?

CORRECT ANSWER



When the notebook crosses ~200 lines, or cell order starts mattering

WHY

Notebooks excel at exploration. They start to fail you when execution order matters, when code is duplicated across cells, or when reviewers need to read it sequentially. At that point, extract pure functions to a .py file and import them back.

``git push`` returns a 'merge conflict'. What do you do?

A

Force-push with `--force` to overwrite the conflict

B

Delete the repo and start over

C

Pull, resolve the conflict locally, commit, then push

D

Email the teammate and ask them to fix it

`git push` returns a 'merge conflict'. What do you do?



CORRECT ANSWER

Pull, resolve the conflict locally, commit, then push

WHY

Conflicts are routine in any team. Pull the remote changes, open the conflicting file in VS Code (it shows a clear three-way merge UI), pick which version wins, commit, push. --force is a last resort that almost always destroys someone's work.

REFLECT & APPLY

Three questions before you close this lesson

Spend 60 seconds on each. Write the answers down — no scrolling, no shortcuts.

Q1

What part of your current setup feels most fragile? venv, dependencies, Git, IDE — pick one and improve it tonight.

Q2

Which keyboard shortcut would save you the most time, and how often do you currently use it?

Q3

Which of your past projects could you re-create on a brand-new laptop in 5 minutes? If none — what's missing?

Ship a public repo. Green push. Pull request. Merge.

Pick any small Python project — even a 30-line script. Set up the full environment by hand, push to a public repo, invite a classmate, accept their pull request, and merge. End-to-end.

DELIVERABLES

- Public GitHub repo with green 'push successful' status
- requirements.txt at the root, .venv in .gitignore
- README with the three commands needed to reproduce your setup
- One classmate added as a collaborator and one PR merged
- A 60-second screen recording of the green push moment

WHAT 'GOOD' LOOKS LIKE

A great submission can be cloned, run, and contributed to by a stranger in under five minutes. That is the standard your future internship will assume on day one.

SUMMARY

Five sentences to remember

If you remember nothing else from this lesson, remember these five lines.

- 1 One venv per project — every project, every machine, no exceptions.
- 2 requirements.txt is the contract that lets anyone reproduce your setup.
- 3 Use Jupyter for exploration, VS Code for engineering, Git for memory.
- 4 Six VS Code extensions — Python, Jupyter, GitLens, Ruff, Error Lens, an AI assistant.
- 5 Your environment is your instrument. Tune before every performance.

END OF LESSON 5

What you can now do

Each one of these is now part of your working vocabulary.

- 1 Spin up a virtual environment + requirements.txt in under a minute.
- 2 Configure VS Code with the six extensions that actually matter.
- 3 Use Jupyter for exploration and know exactly when to leave it.
- 4 Push a public repo from cold start to green status in five minutes.
- 5 Resolve a merge conflict in VS Code without breaking a sweat.

NEXT → LESSON 6 — Hyperautomation Fundamentals